

Pawan: Locally Calibrated Next-Day Air Quality Forecasting for Patiala

Prototype stage: a forecast issued every morning, and an honest account of how good it is

Progress Report for UCS503P

Submitted to: Dr. Paramveer Kaur

Thapar Institute of Engineering and Technology

Gurkirpa Singh, CSED* Ketubh Bansal, CSED[†] Aayush Kandhol, CSED[‡]

12 October 2026

At a glance. Pawan has issued a next-day PM_{2.5} forecast for the Model Town station every morning since 7 September, without anyone running it. It is checked against what the station later measures, and the record is public at [the project page](#).

Over a walk-forward backtest of 488 days it beats the honest baseline, persistence from the freshest reading available at issue time, by **28.5%** in the calm season and **36.5%** in the burning season. The proposal's targets were 10% and 20%. It does not beat persistence from *yesterday's* reading, which nobody has at issue time, and it does not see pollution spikes coming: it flagged none of the 9 days that reached the *Poor* band. Both are reported below rather than left out, and the second sets the direction for the next iteration.

1 Where the project stands

... the short version

The proposal promised three things: a pipeline that runs on its own, an evaluation that cannot flatter itself, and a correction that beats persistence by a stated margin. All three exist.

- **The pipeline runs unattended.** A scheduled job collects the station's observations, the CAMS global forecast, forecast weather and satellite fire detections every morning, refits the model, issues tomorrow's forecast, rebuilds the project page and commits all of it to the repository. The table now holds 589 days, from 19 February 2025 onwards.
- **The evaluation is walk-forward and leakage-checked.** Every scored forecast comes from a model fitted only on days before it, using only what had been published by 09:00 the day before. Forecasts, once issued, are frozen.
- **Both accuracy targets are met** against operational persistence, on 425 calm-season days and 63 burning-season days, including the whole of the 2025 season.

What is not yet good enough is equally definite. The model is a good forecaster of an ordinary day and a poor forecaster of a bad one: its largest prediction across the whole backtest is 75 $\mu\text{g}/\text{m}^3$, and the worst day on record reached 141. Section 6 sets out the numbers and Section 8 the plan.

*Roll No: 1024170378, Email: gsarao.be24thapar.edu

[†]Roll No: 1024170399, Email: kketubh1.be24thapar.edu

[‡]Roll No: 1024170379, Email: akandhol.be24thapar.edu

2 Proposal against prototype

... what changed and why

Proposed	Status	Note
Ingestion adapters for observations, forecast and fires	done	One adapter per source, each validating response content rather than status codes.
Scheduled daily record carrying forecast issue time	done	08:00 IST via GitHub Actions; 57 rows carry a recorded issue date.
Baselines over a fixed period	done	Persistence twice, climatology, raw CAMS. See Section 6.1.
A correction that beats persistence	done	Against operational persistence; not against textbook persistence.
Gradient boosted trees, linear model alongside	changed	Ridge regression only, written by hand. Reasons in Section 5.
Continuous integration on every change	done	Lint and <code>pytest</code> (100 tests) gate every pull request and every data commit.
Published dashboard	done	Forecast and results pages, regenerated from the data on every run.
Fire activity and wind features	done	Lagged fire counts and wind components, in the model since week 5.
Band classification and advisory rule	partly	Bands shown and scored. The advisory is measured and fails; Section 6.4.
Rolling-origin backtest over a full season	done	488 days, 4 April 2025 to 7 October 2026.
Scheduled retraining	done	The model is refitted from scratch every morning.
Feature importance	done	Standardised weights published on the results page.

Table 1: Items from the proposal's scope, as they stand.

3 The system as built

... engineering focus

3.1 Architecture

Figure 1 is the whole system. There is no server and no database. The repository is the system of record: the dataset is a CSV file in git, every change to it is a commit, and the public page is built from the same files the model reads.

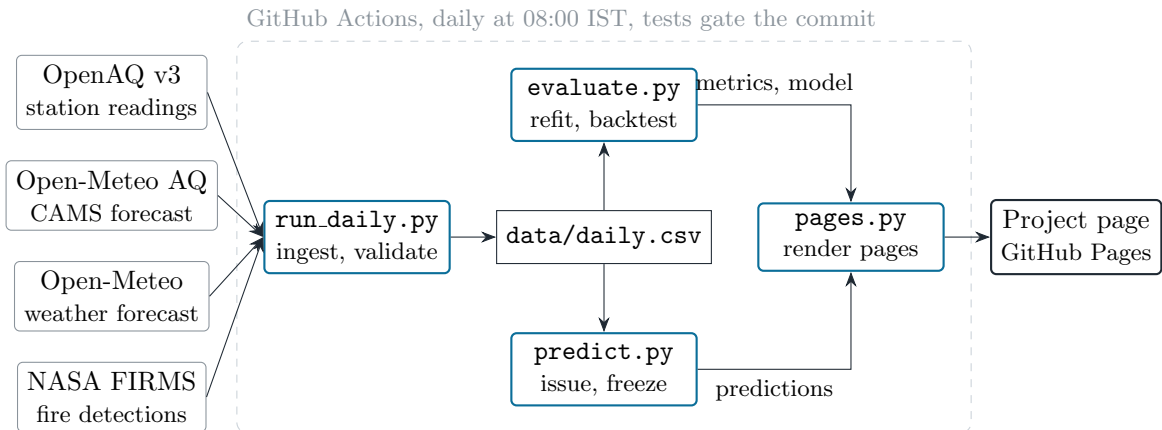


Figure 1: Data flow. Grey boxes are external services, blue boxes are the pipeline stages, and the dashed boundary is what runs unattended each morning. `predict.py` applies the model that `evaluate.py` has just refitted and saved.

3.2 Components

Module	Responsibility
<code>config.py</code>	Every setting in one place: station, sensor ids, lags, validity rules, paths.
<code>ingest/run_daily.py</code>	Fetches a rolling 30-day window of observations and tomorrow's forecast; fails the run if the observation feed has gone stale.
<code>features.py</code>	Turns the table into model rows, and is where the information-set rules live.
<code>model.py</code>	Ridge regression on standardised features, solved by hand; about eighty lines, standard library only.
<code>evaluate.py</code>	Walk-forward backtest, all baselines, the per-season and live-only splits; writes <code>metrics.json</code> and <code>model.json</code> .
<code>predict.py</code>	Issues tomorrow's forecast once, never rewrites it, attaches the outcome when it publishes.
<code>pages.py</code>	Renders the forecast and results pages from the JSON files. Nothing on the site is typed by hand.
<code>scripts/</code>	One-off tools: history extension, archive backfill, model comparison, the figures in this report.

Table 2: `code/` is 2,157 lines; `code/tests/` is a further 1,153.

3.3 Rules the pipeline keeps

- **Standard library only at run time.** The ingest and the model import nothing outside Python. `requirements.txt` holds `pytest` and `ruff` and nothing else, so the daily job cannot break on a dependency upgrade.
- **A forecast is written once.** `predict.py` never rewrites a past prediction, even after the model is refitted. Outcomes are filed beside it when the observation lands.
- **Failures are loud.** A run that issues no forecast, or finds the freshest observation more than seven days old, exits with an error and turns the workflow red. Both rules exist because of a real silent failure (Section 7).
- **Generated, never typed.** The public pages and every number in this report are produced from `data/` by code, so they cannot drift from the record.

3.4 Continuous integration and delivery

Four workflows run the project. **CI** lints and tests every pull request. **Daily ingest** runs the pipeline each morning, validates what it wrote with the full test suite, and only then commits. **mkdocs** rebuilds and deploys the site on every push to **master**. **Extend history** is a manual job, dry-run by default, used once in this iteration to pull the record back to the start of the live sensor. The daily job and the extension share a concurrency group, so they never write the table at the same time. **master** is protected and changes reach it through pull requests; sixteen have been merged so far.

4 Data

... and what was known when

4.1 Sources

Source	What it gives	Lag at 09:00
OpenAQ v3, sensor 12235142	Daily mean $\text{PM}_{2.5}$ at Model Town, with the count of hours behind it. The target.	about 3 days
Open-Meteo Air Quality	CAMS global $\text{PM}_{2.5}$ forecast for tomorrow, daily mean. The thing being corrected.	issued that morning
Open-Meteo Forecast	Temperature, humidity, wind speed and direction for tomorrow.	issued that morning
NASA FIRMS	VIIRS fire detections in an upwind box over Punjab and Haryana.	about 1 day

Table 3: The four inputs. The observation lag is the constraint that shapes everything else.

4.2 The information set

The forecast for day D is issued on the morning of $D - 1$. The station publishes about three days late, so the freshest reading available then is roughly $D - 4$. Figure 2 shows what each input contributes. Every training row is built with the same rules as the live forecast, so the model is never fitted on information it would not have had.

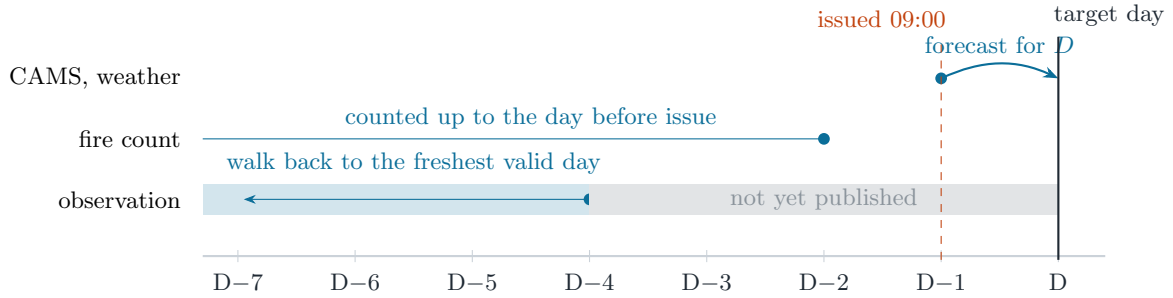


Figure 2: What the forecast for day D may use. A dot is the newest value of each input the forecast is allowed to see. Observations walk back up to three further days when the cutoff day has not been published or is too thin to count.

4.3 Quality rules

- **Sixteen hours or it does not count.** CPCB treats a daily mean built from fewer than 16 hourly readings as invalid, and so does Pawan: such days are kept in the table but never used as a label or as a later day's persistence. 48 of 585 observed days fall below the line.
- **Thin days are refreshed.** The feed publishes a day as soon as it has some hours and fills the rest in later; the ingest replaces a thin reading when more hours arrive.
- **The freshest valid reading, not a fixed day.** The feed lands in batches every two to five days. Reading only $D - 4$ left ten of the first thirty-one target days without a forecast; the reader now walks back up to three further days.

4.4 The history, and a mistake worth recording

Until this iteration the table began on 15 May 2026, because the first bootstrap requested `past_days=92`, on the belief that 92 days was as far back as the CAMS archive went. It is only the ceiling on that one parameter; an explicit start and end date reach back to August 2022.

The live sensor has reported since February 2025, so the 2025 burning season had been available from the first day of the project, and the model had been fitted on summer and monsoon alone.

The consequence was visible in October. On 3 and 5 October the live forecast was $22 \mu\text{g}/\text{m}^3$ against an observed 38: the model had never seen the season starting. Extending the table to 19 February 2025 added every day of October and November 2025, with no gaps. Refitted on that history, the backtest forecasts for the same two days are 33 and 42. Rows from archives carry no issue date and are marked as such; they are training history, never part of the live record.

5 Method

...deliberately ordinary

The model is a ridge regression, refitted from scratch every morning, predicting $\log(1 + y)$ where y is tomorrow's daily mean $\text{PM}_{2.5}$. Fitting in log space makes the penalty proportional, which is how the error is felt: ten out on a reading of twenty matters, ten out on two hundred matters much less. The penalty strength is chosen inside each fit on the most recent fifth of its own training window, so it never sees the day being predicted.

Feature	Meaning	Weight
<code>cams_log</code>	log of CAMS's forecast for tomorrow	+0.197
<code>obs_recent_log</code>	log of the freshest valid observation	+0.113
<code>burning</code>	1 in October and November	+0.037
<code>temp, rh</code>	forecast temperature and humidity	-0.159, -0.252
<code>wind.speed</code>	forecast wind speed	-0.124
<code>wind.u, wind.v</code>	wind as eastward and northward components	+0.042, +0.023
<code>fire_log</code>	log of the upwind fire count the day before issue	+0.116

Table 4: The nine features and their standardised weights in the model fitted on 10 October. Signs read physically: humid, warm and windy days are cleaner; fires and a high CAMS forecast push the estimate up.

Why not gradient boosting. The proposal planned a boosted tree ensemble with a linear model alongside. Two things changed the plan. Until this iteration there were never more than about a hundred usable rows, which is too few for trees to beat a penalised linear fit. And the daily job runs unattended until the end of the semester, which made a standard-library-only pipeline worth more than a library dependency. The linear model was therefore built first, and it is what is reported. With 528 usable rows now on hand, a boosted model is worth testing offline, and it is in the plan.

Candidates tested and not adopted. With a burning season in the table, four ways of letting the season change the model were backtested against the current one: season interactions on the fire count and the eastward wind (smoke transport), on persistence and CAMS (how far to trust each), all four together, and a separate fit for burning-season days. The bar was set before the comparison: clearly better in burning season, no more than a point or two worse in calm. None cleared it. The best, smoke transport, gained 1.3 points in burning season and lost 1.7 on the live days, which on 63 days is noise. The model was left as it is.

6 Evaluation

...including the awkward numbers

6.1 Baselines

- **Operational persistence:** tomorrow equals the freshest reading published by 09:00, about four days old. The same information Pawan has, so the fair comparison and the bar the targets are measured against.

- **Textbook persistence:** tomorrow equals today. The baseline the proposal quoted. Nobody has today’s reading when the forecast goes out, so it marks a ceiling rather than a competitor: the gap between the two persistences is the cost of the publication lag, and closing it needs a faster feed rather than a better model.
- **Raw CAMS:** the uncorrected global forecast.

6.2 Results

	Days	Pawan	Operational	Textbook	Raw CAMS	Skill
Calm season	425	8.11	11.35	7.12	43.89	+28.5%
Burning season	63	12.16	19.14	11.32	28.21	+36.5%
Live rows only	47	4.33	7.50	4.17	41.17	+42.3%
All	488	8.63	12.36	7.66	41.87	+30.1%

Table 5: Mean absolute error in $\mu\text{g}/\text{m}^3$, walk-forward, 4 April 2025 to 7 October 2026. Skill is the reduction against operational persistence; the targets were 10% calm and 20% burning. Live rows are those whose CAMS forecast was recorded on the day rather than read back from an archive.

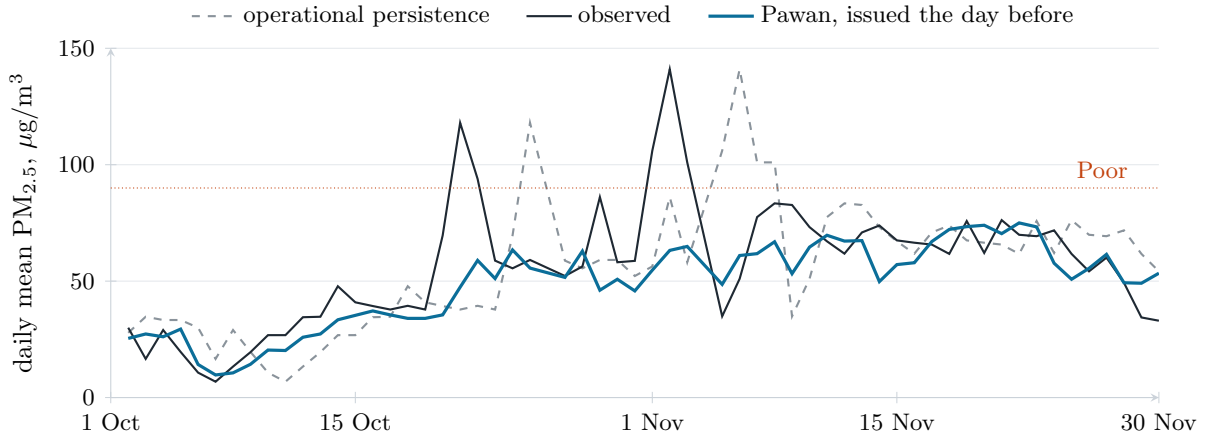


Figure 3: The 2025 burning season, day by day, as the walk-forward backtest forecast it. Pawan follows the season’s level far better than a four-day-old reading does, and tops out well short of the peaks.

Three things in Table 5 are worth saying directly. Both targets are met with room to spare, in the burning season on a full season of data. The live rows, which no archive has helped, score better than the backfilled ones, though they cover only the last two months, so the archive’s mild optimism is not what is carrying the result. And Pawan loses to textbook persistence in every row, by 13.8% in the calm season and 7.4% in the burning one. Most of what the model adds is recovering what the four-day lag throws away; it is not better than knowing yesterday.

6.3 What the correction is correcting

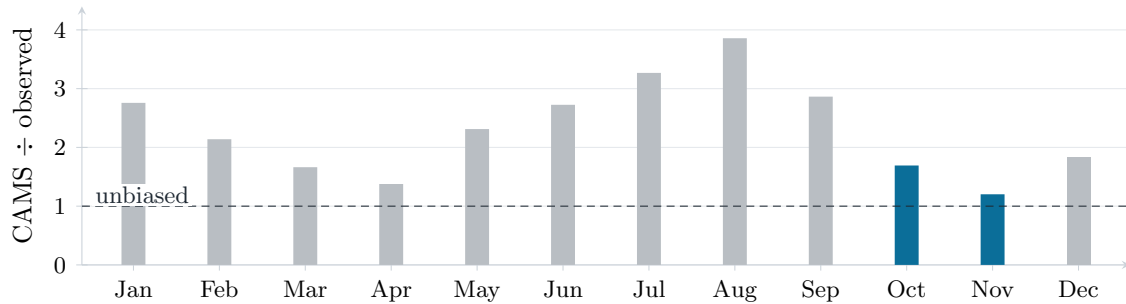


Figure 4: How far CAMS overshoots, by month, February 2025 to October 2026. It reads 3.9 times the observed value in the monsoon and 1.2 times in November (burning months in blue). A single scale factor cannot correct both, which is why the correction depends on weather and season.

6.4 What it gets wrong

- **It does not see spikes.** Of the 9 days that reached the *Poor* band, Pawan flagged none; its highest forecast anywhere in the backtest is $75 \mu\text{g}/\text{m}^3$. On the five worst days it forecast, on average, 44% of what was observed; on 2 November 2025 the station read 141 and the forecast was 63. One band lower, of 63 days worse than *Satisfactory* it caught 17, against 35 for operational persistence. Part of the gain in average error comes from forecasting towards the middle, which is the right trade for an ordinary day and the wrong one for an advisory.
- **The likely range is too narrow.** The published band is meant to hold about four days in five. On the 18 scored live forecasts it held 10.
- **The live service missed days.** Between 7 September and 11 October it issued 24 of 35 forecasts. Most of the misses came from the fixed-day observation rule; the fix was merged on 10 October.
- **Backfilled rows are mildly optimistic.** Archive CAMS and weather are stitched from the opening hours of successive model runs. The live-only row in Table 5 is the check on this, and so far it agrees.

7 Engineering log

...found, fixed, guarded

Every fault below was found in the running system, fixed through a pull request, and left behind a test or a run-time check so that it cannot come back quietly. The weekly journals tell each one in full.

Week	What went wrong	Guard left behind
2	The observation API ignored misspelt date filters and returned the whole series, which made a sensor dead since 2022 look like the better one.	Dates re-checked client side; the live sensor id pinned in config with the reason.
3	The persistence baseline in the proposal used a reading that does not exist at issue time.	Two persistences reported side by side; a test that features never see past the publication lag.
4	The ingest called the wrong endpoint and committed nothing new for three weeks while reporting success.	Staleness tripwire: the run fails if the freshest reading is over seven days old.
5	The model lost to persistence by 22.6%, from leaked fire counts, a seasonal term acting as a trend, and thin days used as labels.	Fire count lagged; sixteen-hour rule; tests for each.
Oct	Ten of thirty-one target days had no forecast; the page showed a stale one as tomorrow's; a clock-dependent test would have blocked data commits from 22 October.	Walk-back reader; status block and honest empty card; test dates pinned.
Oct	The history began in May because a request parameter limit was mistaken for the archive's.	Extension script and a dry-run workflow; <code>bootstrap.py</code> refuses to overwrite the table.

Table 6: Faults found in operation.

8 Improvement plan

... week 8 onwards

In priority order. Each item has a test that says whether it worked, decided before it is tried.

1. **See the spikes.** Add an exceedance forecast alongside the point forecast, the probability that tomorrow is worse than *Satisfactory*, and correct the log-space back-transform, which currently returns something closer to a median than a mean. *Done when* recall on days above $60 \mu\text{g}/\text{m}^3$ beats operational persistence's 35 of 63 without more false alarms than it.
2. **Calibrate the likely range** from the backtest's own residuals by season, instead of one spread for the year. *Done when* the band holds between 75 and 85% of days.
3. **Score the 2026 season live.** October and November are the test nothing here was tuned against. The results page already separates live rows; the final report will lead with them.
4. **A second burning season in April and May.** The fire box sees more detections during wheat residue burning than in November, but the **burning** flag covers only October and November. Test a flag for both, by the same pre-set bar as the candidates above.
5. **Try boosted trees offline**, now that there are 528 usable rows, with the linear model as the reference. Adopted only if it wins on both seasons and the live rows.
6. **An interactive layer** on the project page, if time allows after the above.

9 Risks

- **One season is one season.** The burning-season result rests on 2025 and the first days of 2026. A wetter or drier year, or a shift in when the stubble is burnt, would move it.
- **The feed lag sets a ceiling.** Pawan cannot beat a reading it does not have. A faster observation source would do more for accuracy than any model change.

- **Upstream change.** CAMS is upgraded from time to time and its bias could shift between seasons. The live-only score is the early warning.
- **Platform drift.** GitHub moves its default runner to a new Ubuntu release on 19 October. The workflows are pinned to the current one so the daily job does not change underneath the project before the final seminar.

10 Reproducing this report

Every number above is generated from the repository's data. From a fresh clone:

```
pip install -r requirements.txt
python code/evaluate.py
python scripts/report_figures.py
cd project-report-prototype-stage
latexmk -pdf main.tex
```

The figures here are a snapshot of the record on 10 October 2026. The results page on the project site is regenerated every morning and is the current one.